

UD 9. CONTROL Y MANEJO DE EXCEPCIONES

**Programación – 1º DAM
Curso 2025/2026**

Herencia

¿Qué es una excepción?

Jerarquía de excepciones

Manejo de excepciones

Lanzar excepciones con throw y throws

Creación de excepciones personalizadas

¿Qué es una excepción?

Una excepción es un problema que surge durante la ejecución de un programa. Una excepción puede ocurrir por muchas razones diferentes, por ejemplo:

- Un usuario ha introducido datos no válidos.
- Un archivo que necesita ser abierto no se puede encontrar .
- Una conexión de red se ha perdido en el medio de las comunicaciones o la JVM se ha quedado sin memoria.
- Etc...

Algunas de estas excepciones son causadas por error del usuario, otras por error del programador, y otras por los recursos físicos que han fallado de alguna manera.

Jerarquía de excepciones

Las excepciones en Java forman una jerarquía bajo la clase base **Throwable**:

Throwable

└─ **Exception** (Excepciones verificadas y no verificadas)

│ └─ IOException

│ └─ SQLException

│ └─ **RuntimeException** (Excepciones no verificadas)

│ └─ NullPointerException

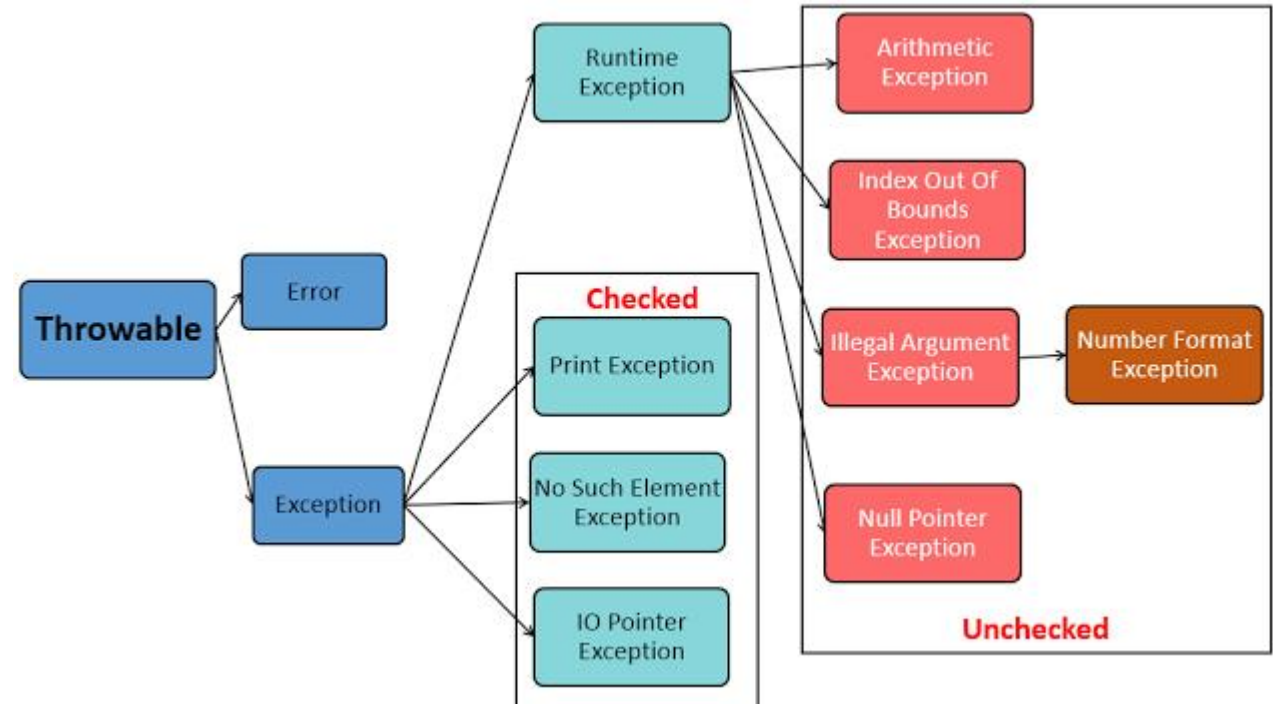
│ └─ IndexOutOfBoundsException

│ └─ ArithmeticException

└─ **Error** (Errores críticos)

└─ StackOverflowError

└─ OutOfMemoryError



Jerarquía de excepciones

Podemos ver en la clasificación anterior 3 tipos:

- **Checked Exceptions (Verificadas):** son excepciones ajenas al código, pero inherentes a él, como por ejemplo intentar acceder a un fichero que no existe. Se deben manejar obligatoriamente con try-catch o throws.
- **Unchecked Exceptions (No verificadas):** en este caso el fallo sí que se encuentra en el código, como, por ejemplo, acceder a una posición fuera de los límites de un array, asignar un dato de tipo incorrecto, etc. Son errores en tiempo de ejecución y no requieren manejo obligatorio.
- **Error (Errores):** Son fallos graves que no se pueden manejar en la mayoría de los casos. Suelen ser errores producidos por hardware como, por ejemplo, poco espacio en disco, problemas con la memoria, etc.

Manejo de excepciones

USO DE try-catch-(finally)

Bloque TRY. Todo el código que vaya dentro de este bloque será el código sobre el que se intentará capturar el error si se produce y una vez capturado hacer algo con él. Lo ideal es que no ocurra un error, pero en caso de que ocurra un bloque try nos permite estar preparados para capturarlo y tratarlo. Así un ejemplo sería:

```
try {  
    System.out.println("bloque de código donde pudiera saltar un error es este");  
}
```

Manejo de excepciones

USO DE try-catch- (finally)

Bloque CATCH. Se usa para manejar excepciones cuando ocurren dentro del bloque try. Si una excepción ocurre, el flujo del programa salta al catch, donde se puede gestionar el error.

Si ocurre un error en try, el catch lo captura y evita que el programa termine abruptamente.

```
catch (Exception e) {  
    System.out.println("bloque de código donde se trata el problema");  
}
```

Manejo de excepciones

USO DE try-catch-(finally)

Bloque FINALLY. Se ejecuta siempre, haya ocurrido o no una excepción en el bloque try. Se usa para limpiar recursos, cerrar conexiones o ejecutar código que debe correr independientemente de si hubo un error.

En Java, la palabra clave finally se utiliza en combinación con los bloques try y catch para manejar excepciones y asegurar que cierto código se ejecute, independientemente de si se produce una excepción o no

```
finally {  
    System.out.println("bloque de código ejecutado siempre");  
}
```


Manejo de excepciones

USO DE try-catch-(finally)

Ejemplo

```
try {  
    int[] numeros = {1, 2, 3};  
    System.out.println(numeros[5]); //Error: índice fuera de  
    rango  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Error: Intentaste acceder a un índice  
    inválido.");  
} finally {  
    System.out.println("Este mensaje siempre se ejecutará.");  
}
```

Manejo de excepciones

RESUMEN

Ejecutamos las instrucciones del bloque try que no dan errores, pero cuando en una instrucción se produce un error o excepción inesperada se deja de ejecutar el código del bloque try, y pasamos a ejecutar el código del bloque catch. Hay un salto o cambio en el flujo del programa.

Finalmente se ejecutan, en todo caso, las instrucciones del bloque finally.

El bloque finally no es obligatorio, es decir, puede existir un bloque try catch y no existir bloque finally.

Manejo de excepciones

USO DE try-with-resources

Liberación automática de recursos. Es una característica introducida en Java 7 que cierra automáticamente los recursos al final del bloque try, sin necesidad de hacerlo manualmente en finally.

Cuando usamos recursos como archivos, conexiones a bases de datos o sockets, es necesario cerrarlos después de usarlos para evitar problemas de fugas de memoria o bloqueos de archivos.

Antes de Java 7, teníamos que cerrar manualmente los recursos usando finally. Esto hacía que el código fuera más largo y propenso a errores.

Manejo de excepciones

USO DE try-with-resources

Ejemplo

```
public class EjemploTryWithResources {  
    public static void main(String[] args) {  
        try(BufferedReader br=new BufferedReader(new FileReader("archivo.txt"))){  
            System.out.println("Contenido: " + br.readLine());  
        catch (IOException e) {  
            System.out.println("Error al leer el archivo: " +e.getMessage());  
        }  
    }  
}
```

Manejo de excepciones

Esa mejora propuesta en la versión 7 tenía una limitación, que el recurso debía declararse dentro del try, si había sido creado antes no podía usarse.

Desde Java 9 puedes usar una variable ya creada dentro del try. La única condición es que sea final o effectively final (no se modifica después).

Una variable es **effectively final** si solo se asigna una vez, es decir, que no podemos reutilizar ese nombre de variable después.

```
BufferedReader br = new BufferedReader(new FileReader("archivo.txt"));
```

```
try (br) {  
}
```



```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));  
br = new BufferedReader(new FileReader("b.txt")); // cambia el valor
```

```
try (br) {  
}
```



Manejo de excepciones

Por tanto, quedaría de la siguiente forma:

En Java 7 habría que hacerlo así:

```
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {  
}
```

Y a partir de Java 9, ya podría hacerse así:

```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));  
try (br) {  
}
```

Lanzar excepciones con throw y throws

throw (lanzar una excepción manualmente)

Usamos **throw** para generar una excepción **en tiempo de ejecución**. Puede suceder que queramos que nuestro código lance una excepción en una situación determinada.

Su sintaxis es:

```
throw new TipoDeExcepcion("Mensaje de error");
```

- La palabra clave **throw** va seguida de una instancia de una clase de excepción.
- Puedes usar cualquier clase que herede de **Throwable** (como **Exception** o **RuntimeException**).
- Después de **throw**, el programa se detiene inmediatamente, a menos que se maneje con try-catch.

Lanzar excepciones con throw y throws

throw (lanzar una excepción manualmente)

Ejemplo

```
public class EjemploThrow {  
    public static void validarEdad(int edad) {  
        if (edad < 18) {  
            throw new IllegalArgumentException("Debes ser mayor de edad.");  
        }  
        System.out.println("Acceso permitido.");  
    }  
    public static void main(String[] args) {  
        validarEdad(16); // Esto lanza una excepción  
    }  
}
```


Lanzar excepciones con throw y throws

throws (propagar excepciones)

El método que tiene en su cabecera **throws** indica que puede lanzar una excepción y que debe ser manejada por quien lo llama.

para generar una excepción **en tiempo de ejecución**. Puede suceder que queramos que nuestro código lance una excepción en una situación determinada.

Su sintaxis es:

```
cabeceraMétodo throws nombreException{  
    ...  
}
```

Lanzar excepciones con throw y throws

throws (propagar excepciones)

Ejemplo

```
public class EjemploThrows {  
    public static void leerArchivo(String nombArc) throws IOException {  
        BufferedReader br = new BufferedReader(new FileReader(nombArc));  
        System.out.println(br.readLine());  
        br.close();  
    }  
    public static void main(String[] args) {  
        try {  
            leerArchivo("archivo.txt");  
        } catch (IOException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

Creación de excepciones personalizadas

En Java, podemos crear **excepciones personalizadas** para manejar errores específicos en nuestras aplicaciones. Para ello, aprovecharemos dos de las características más importantes de Java, la herencia y el polimorfismo.

Para hacerlo seguiremos los siguientes pasos:

1. Crear una clase que extienda `Exception` o `RuntimeException`.
2. Crear un constructor que reciba un mensaje.
3. Usar `throw` para lanzar la excepción personalizada.
4. (Opcional). Crear constructores adicionales.
5. Crear excepciones no verificadas (`RuntimeException`).

Creación de excepciones personalizadas

1. Crear una clase que extienda **Exception** o **RuntimeException**

Para crear nuestra propia excepción debemos **extender** la clase de la clase **Exception** o **RuntimeException** de Java.

- **Exception** → para excepciones verificadas (checked exceptions).
- **RuntimeException** → para excepciones no verificadas (unchecked exceptions).

Su sintaxis es:

```
class ExcepcionPropia extends Exception{  
    ...  
}  
class ExcepcionPropia extends RuntimeException{  
    ...  
}
```

Creación de excepciones personalizadas

2. Crear un constructor que reciba un mensaje

El constructor de la excepción debe llamar a `super(mensaje)`, que define el mensaje de error.

Ejemplo

```
// Paso 1: Extender de Exception (Checked Exception)
class EdadInvalidaException extends Exception {
    public EdadInvalidaException(String mensaje) {
        super(mensaje); // Mensaje personalizado
    }
}
```

Creación de excepciones personalizadas

3. Usar throw para lanzar la excepción personalizada

Tenemos que localizar aquella línea de código (o líneas) en las que se detecta una condición inválida. El método en el que se encuentre dicha línea (o líneas), lanzará la excepción.

Ejemplo

Creación de excepciones personalizadas

3. Usar throw para lanzar la excepción personalizada

```
public class ValidarEdad {  
    // Método que lanza la excepción personalizada  
    public static void verificarEdad(int edad) throws EdadInvalidaException {  
        if (edad < 18) {  
            throw new EdadInvalidaException("Edad inválida: debes ser mayor de 18 años.");  
        }  
        System.out.println("Edad válida. Acceso permitido.");  
    }  
  
    public static void main(String[] args) {  
        try {  
            verificarEdad(16); // Intentamos validar una edad inválida  
        } catch (EdadInvalidaException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

Creación de excepciones personalizadas

4. (Opcional). Crear constructores adicionales

Podemos agregar más constructores para personalizar aún más la excepción.

Ejemplo

```
class EdadInvalidaException extends Exception {  
    public EdadInvalidaException(String mensaje) {  
        super(mensaje);  
    }  
  
    public EdadInvalidaException(String mensaje, Throwable causa) {  
        super(mensaje, causa); // Excepción con causa  
    }  
}
```


Creación de excepciones personalizadas

5. Crear excepciones no verificadas (RuntimeException)

En este caso, no necesitaríamos propagar la excepción, es decir, **no necesitaríamos throws en la firma del método ni try-catch** obligatorio.

Ejemplo

```
class NombreInvalidoException extends RuntimeException {  
    public NombreInvalidoException(String mensaje) {  
        super(mensaje);  
    }  
}  
  
public static void verificarNombre(String nombre) {  
    if (nombre.isEmpty()) {  
        throw new NombreInvalidoException("El nombre no puede ser vacío");  
    }  
}
```